23CSE111 OBJECT ORIENTED PROGRAMMING
S2 B.Tech CSE
Department of Computer Science and Engineering
Amrita School of Computing, Amritapuri Campus
CASE STUDY
PHASE – 2 (Design)
Architectural Modelling (CO2)

Group No : C7

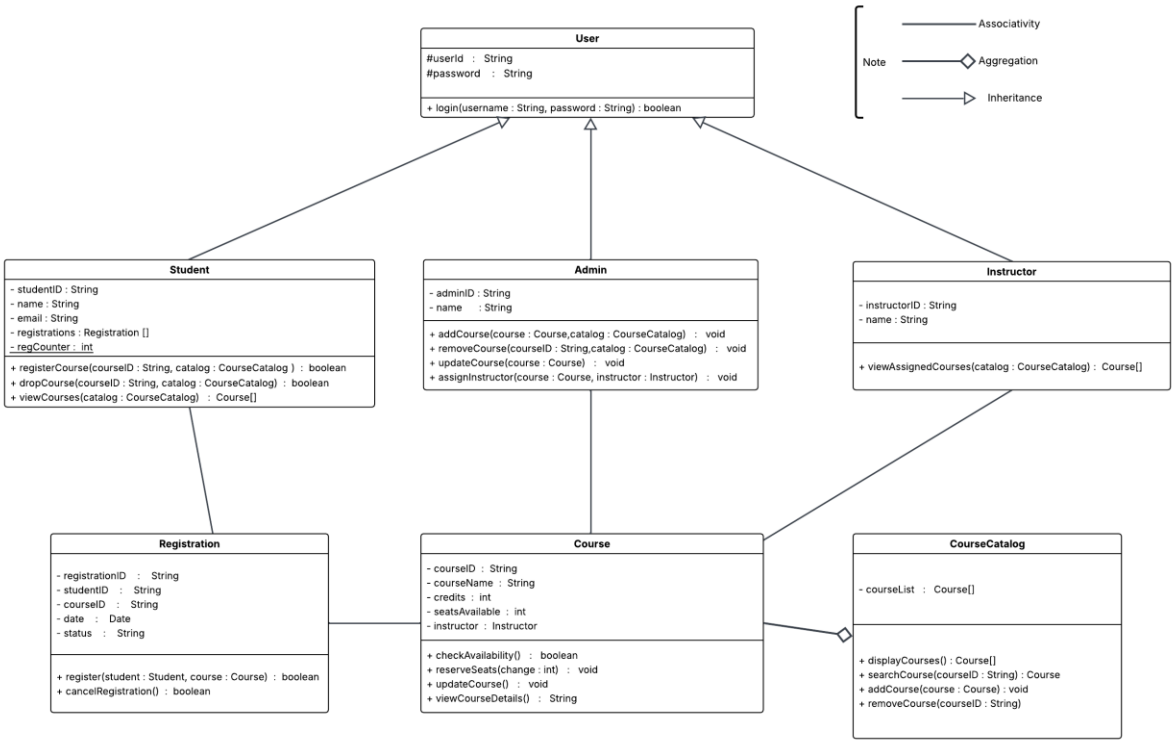
Team Members :

Roll No	Student Name
AM.SC.U4CSE25261	Adithyadev
AM.SC.U4CSE25266	Abhishek R Pillai
AM.SC.U4CSE25260	Dhanesh S
AM.SC.U4CSE25265	Nived Krishna

1. Problem Title : Course Registration System

STRUCTURAL DESIGN

2. Class diagram (*draw class diagram that shows different classes and their relationships. Use UML notations*)



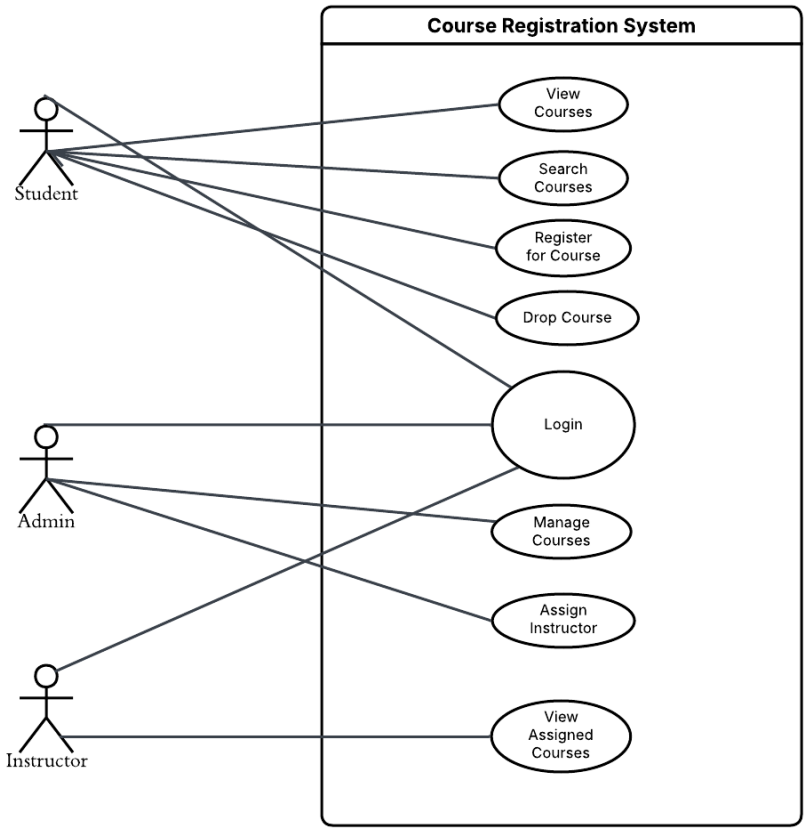
LINK: https://lucid.app/lucidchart/820e2fd0-910e-4f72-abb2-ee785efa1412/edit?viewport_loc=-841%2C-1017%2C4358%2C2609%2C0_0&invitationId=inv_f344a1b3-9755-4e8f-8427-56c8d0e57c45

BEHAVIUORAL DESIGN

3. Use-case diagram
    a. List of actors
        1. Student
        2. Admin
        3. Instructor

    b. List of use-cases
        1. View Courses — Student can view all available courses.
        2. Search Courses — Student can search for a specific course.
        3. Register for Course — Student registers for a course.
        4. Drop Course — Student drops a registered course.
        5. Login — All users (Student/Admin/Instructor) log into the system.
        6. Add Course — Admin adds a new course.
        7. Update Course — Admin updates course details.
        8. Remove Course — Admin removes a course.
        9. Assign Instructor — Admin assigns instructor to a course.
        10. View Assigned Courses — Instructor views courses assigned to them.
        11. Reserve Seats — System updates seats automatically when registration or drop happens.

    c. Use case diagram (use UML notations)



Description:

The Course Registration System includes three actors: Student, Admin, and Instructor.

The Student interacts with the system to view courses, search courses, register for a course, drop a course, and login.

The Admin interacts with the system to login, manage courses (add, update, remove), and assign instructors.

The Instructor interacts with the system to login and view assigned courses.

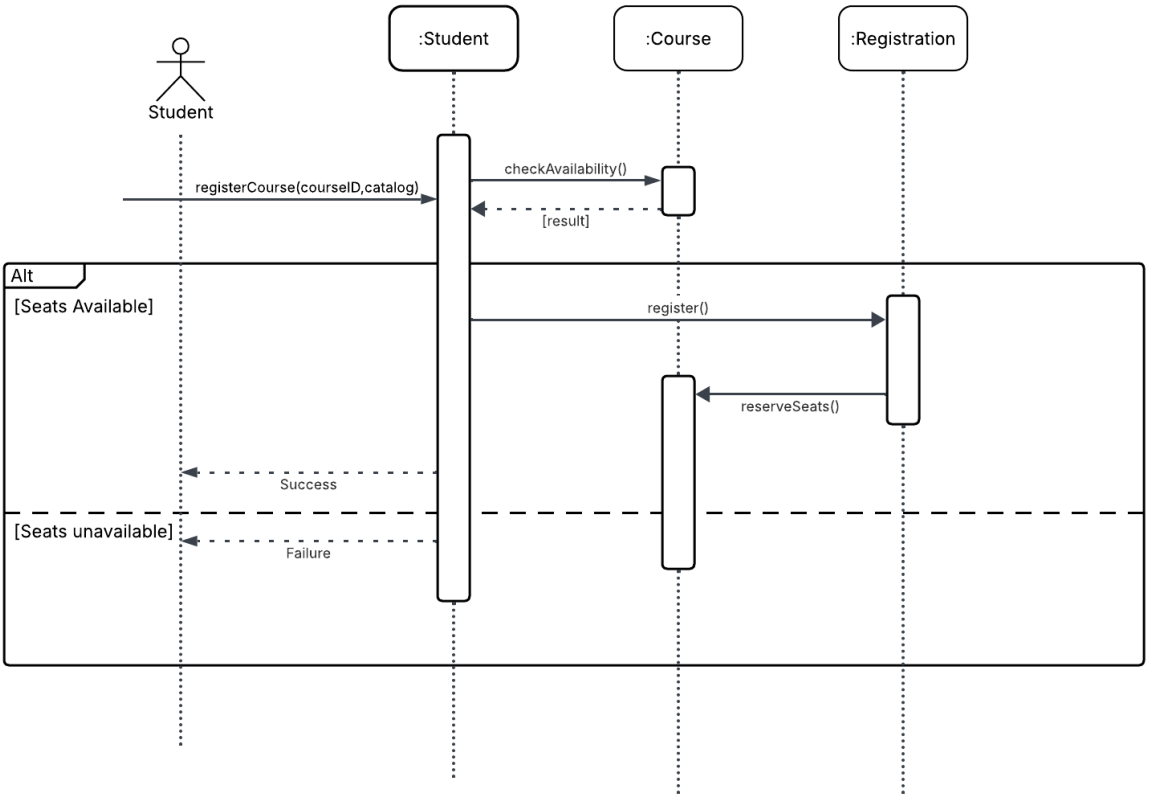
All use-cases are inside the Course Registration System boundary. Each actor is connected to the relevant use-cases they are authorized to perform.

LINK: https://lucid.app/lucidchart/db4a2242-f141-46c2-8c0e-3fb6943100bd/edit?viewport_loc=-1%2C-522%2C2651%2C1574%2C0_0&invitationId=inv_eef90e14-a30b-4e53-ae0e-530b5d9da860

4. Sequence diagram (use UML notations)
   a. Choose key use cases and draw their sequence diagram. First describe the scenario in few sentences. Then draw the sequence diagram.

STUDENT REGISTERS FOR A COURSE

Student chooses to register for a course. The system checks the available seat count for the selected course. If the seat count is less than or equal to 0, the student receives a registration failed message. Otherwise, the system calls reserveSeats() to decrease the seat count and creates a registration entry. The student then receives a message that registration is successful.

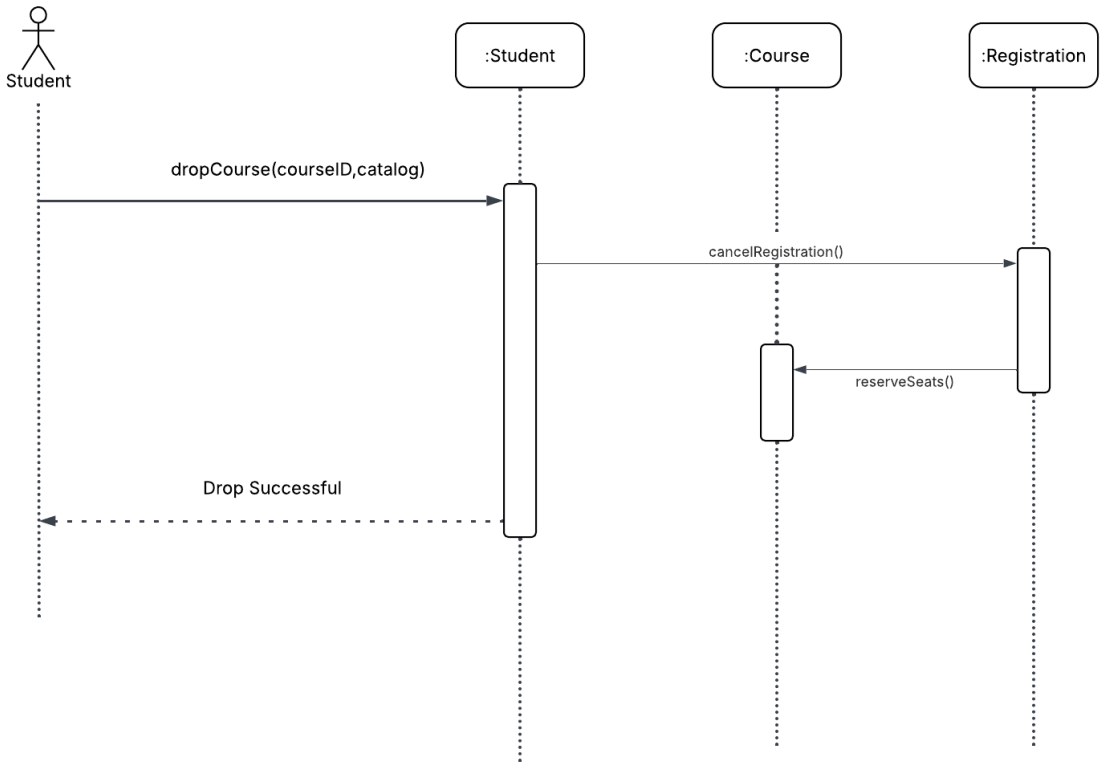


LINK: https://lucid.app/lucidchart/76c7d897-a754-4993-bc41-f26ab1682b78/edit?viewport_loc=-257%2C-69%2C2280%2C1365%2C0_0&invitationId=inv_ce00c765-972e-4ffb-8319-1b4058ebd885

b. Choose key use cases and draw their sequence diagram. First describe the scenario in few sentences. Then draw the sequence diagram.

STUDENT DROPS A COURSE

Student chooses to drop a registered course. The system cancels the registration and updates the seat availability. The system increases the available seat count by calling reserveSeats() . Finally, the student receives a confirmation message that the course has been successfully dropped.



LINK: https://lucid.app/lucidchart/2de0480d-f51b-4c70-8d61-8a079f8dffd0/edit?viewport_loc=-105%2C40%2C2225%2C1332%2C0_0&invitationId=inv_01b3d301-7833-4b1f-8dc5-2c6f968c2d28

Enhancements to the System Design

In this phase, we have enhanced the initial system design by introducing additional components that were not explicitly specified in the Phase 1 . We added an Instructor role and a common User base class to improve structure using inheritance. A login functionality was also included to allow all users to access the system securely.

Furthermore, we incorporated methods such as checkAvailability(), reserveSeats(), registration handling, and assignInstructor() to make the system more realistic and functionally complete. These enhancements provide a clearer representation of the system workflow and improve the overall design quality of the Course Registration System.